

Q1.

You are building a text-formatting utility for a document processing application. One of the required features is converting user-entered text to uppercase and reporting how many words it contains, to help with quick document statistics.

Problem Statement:

Write a program that reads a line of text s and prints the uppercase version of s on the first line, followed by the number of words in s on the second line. Words are separated by single spaces.

Input:

A single line of text s (length ≤ 200), containing one or more words.

Output:

Print the uppercase version of s on the first line, then print the word count on the second line.

Example:

Input:

```
java is fun
```

Output:

```
JAVA IS FUN  
3
```

Q2.

You are developing a data-analysis utility for a weather-tracking system. Temperature readings from multiple sensors are stored in a grid, and the system must quickly identify the highest recorded temperature across the entire grid.

Problem Statement:

Write a program that reads a 2D matrix of integers with r rows and c columns, and prints the largest value present in the matrix.

Input:

The first line contains two integers r and c ($1 \leq r, c \leq 20$). The next r lines each contain c space-separated integers.

Output:

Print a single integer, the largest value in the matrix.

Example:

Input:

```
2 3
```

3 7 2
9 4 1

Output:

9

Q3.

You are building a vehicle-management module for a fleet-tracking application. Different vehicle types calculate fuel efficiency differently, but the system must be able to compute efficiency for any vehicle type through a single common interface, allowing new vehicle types to be added without changing existing code.

Problem Statement:

Design an interface `Vehicle` with a method `double fuelEfficiency(double distance, double fuelUsed)`. Create two classes, `Car` and `Bike`, that implement this interface, both computing efficiency as distance divided by fuelUsed. Write a program that reads the vehicle type (1 for Car, 2 for Bike), the distance travelled, and the fuel used, creates the corresponding object using a `Vehicle` reference, calls `fuelEfficiency()` polymorphically, and prints the result rounded to 2 decimal places.

Input:

The first line contains an integer choice (1 for Car, 2 for Bike). The second line contains two doubles: distance and fuelUsed.

Output:

Print the fuel efficiency (distance / fuelUsed) rounded to 2 decimal places.

Example:

Input:

1
150 10

Output:

15.00

Q4.

You are building a text-analysis utility for a document editor. One of the frequently used features is counting how many times a specific character appears within a block of text, which helps highlight character frequency to the user.

Problem Statement:

Write a program that reads a string `s` (length ≤ 200) and a character `ch`, and prints the number of times `ch` occurs in `s`.

Input:

The first line contains the string *s*. The second line contains a single character *ch*.

Output:

Print a single integer, the count of occurrences of *ch* in *s*.

Example:**Input:**

```
programming  
g
```

Output:

```
2
```

Q5.

You are building a visitor-counter module for a museum's ticketing system. The system must track, using a class-level counter, the total number of visitor objects created since the application started, regardless of how many separate ticket counters are creating them.

Problem Statement:

Design a class *Visitor* that increments a static counter every time an object of the class is created. Write a program that reads an integer *n*, creates *n* objects of the *Visitor* class, and prints the final value of the static counter.

Input:

A single integer *n* ($1 \leq n \leq 1000$), the number of *Visitor* objects to create.

Output:

Print a single integer, the final value of the static counter.

Example:**Input:**

```
4
```

Output:

```
4
```

Q6.

You are building a payroll module for a company's HR system. Different employee types calculate their salary differently: managers receive a fixed bonus on top of their base salary, while developers receive a bonus based on the number of completed projects. The system must compute the correct salary depending on employee type using a common base type.

Problem Statement:

Design a base class Employee with a method double calculateSalary() that returns the base salary unchanged. Create two subclasses: Manager, which adds a fixed bonus of 5000 to the base salary, and Developer, which adds 2000 for every completed project to the base salary. Write a program that reads the employee type (1 for Manager, 2 for Developer), the base salary, and (for Developer) the number of completed projects, then creates the corresponding object using an Employee reference, calls calculateSalary() polymorphically, and prints the result.

Input:

The first line contains an integer choice (1 for Manager, 2 for Developer). The second line contains the base salary (a double). If choice is 2, a third line contains an integer, the number of completed projects.

Output:

Print the calculated salary as a double.

Example:

Input:

1
30000

Output:

35000.0

Q7.

You are building a text-validation utility for a form-processing application. One of the checks required is verifying whether a user-entered word or phrase reads the same forwards and backwards, ignoring case differences.

Problem Statement:

Write a program that reads a string s (length ≤ 100 , no spaces) and prints "Palindrome" if it reads the same forwards and backwards (case-insensitive), otherwise prints "Not Palindrome". Use the StringBuilder class to reverse the string.

Input:

A single string s containing only alphabetic characters.

Output:

Print "Palindrome" or "Not Palindrome".

Example:

Input:

Madam

Output:

Palindrome

Q8.

You are developing a spreadsheet-like utility that stores numeric data in a grid layout. One of the required features is computing the total of all values present in the grid.

Problem Statement:

Write a program that reads a 2D matrix of integers with r rows and c columns, and prints the sum of all its elements.

Input:

The first line contains two integers r and c ($1 \leq r, c \leq 20$). The next r lines each contain c space-separated integers.

Output:

Print a single integer, the sum of all elements in the matrix.

Example:**Input:**

```
2 3
1 2 3
4 5 6
```

Output:

```
21
```

Q9.

You are building a geometry module for a CAD (Computer-Aided Design) application. The module must support multiple shape types and compute the area of any shape polymorphically through a common abstract type, allowing new shapes to be added easily in the future.

Problem Statement:

Design an abstract class Shape with an abstract method `double area()`. Create two subclasses: Circle (with a radius) and Rectangle (with length and breadth), each overriding `area()` with its own formula. Write a program that reads a shape type (1 for Circle, 2 for Rectangle) followed by its dimension(s), creates the corresponding object using the Shape reference, calls `area()` polymorphically, and prints the result rounded to 2 decimal places. Use 3.14 for the value of pi.

Input:

The first line contains an integer choice (1 for Circle, 2 for Rectangle). If choice is 1, the next line contains the radius (a double). If choice is 2, the next line contains two doubles: length and breadth.

Output:

Print the computed area rounded to 2 decimal places.

Example:**Input:**

1
5

Output:

78.50

Q10.

You are developing a Student Management System for a school's admission portal. Every time a new student registers, the system creates a new Student object, and the administration wants the portal to automatically keep track of the total number of students admitted so far, without relying on any external counter.

Problem Statement:

Write a Java program that defines a Student class with a static variable to keep count of the total number of Student objects created. The constructor should increment this static variable every time a new object is created. After creating multiple Student objects, print the total number of students using the static variable.

Input:

An integer n , the number of Student objects to create ($0 \leq n \leq 100$).

Output:

The total number of students created, printed using the static counter.

Example:**Input:**

3

Output

Total Students: 3

Q11.

You are building an animal simulation module for a wildlife education app. The app defines a general Animal type, but each specific animal, such as a Dog or a Cat, behaves differently when asked to make a sound. Based on the animal name typed in by the user, the app should always call the correct, specific behavior at runtime, even when it only holds a reference to the general Animal type.

Problem Statement:

Write a Java program with a base class `Animal` containing a method `sound()` that prints a generic message. Create subclasses `Dog` and `Cat` that each override `sound()` with their own message. Read the name of an animal ("Dog", "Cat", or anything else) from the user, create the matching object using an `Animal` reference, and call `sound()` to demonstrate runtime polymorphism.

Input:

A single string, the animal name: "Dog", "Cat", or any other value.

Output:

The message printed by the overridden `sound()` method of the matching class (or the generic `Animal` message if the name doesn't match `Dog` or `Cat`).

Example:**Input:**

Dog

Output

The dog barks

Q12.

You are building the login module for a college portal. When a student logs in, the system must check whether the username they typed exactly matches the username stored during registration, since even a small case difference should be treated as a mismatch for security reasons.

Problem Statement:

Write a Java program that reads two strings, the registered username and the entered username, and checks whether they are exactly equal using the `equals()` method. Print "Match" if the two strings are identical, otherwise print "No Match".

Input:

Two strings on separate lines: the registered username and the entered username.

Output:

"Match" if both strings are exactly equal, otherwise "No Match".

Example:**Input:**

admin

admin

Output

Match

Q13. Vehicle Interface Implementation

You are building a vehicle rental system that supports multiple vehicle types, such as Car and Bike. The system needs every vehicle type to guarantee the same set of actions (starting and stopping), so the rental app can operate on any vehicle uniformly through a common contract.

Problem Statement:

Define an interface Vehicle with abstract methods start() and stop(). Implement this interface in two classes, Car and Bike, each providing its own version of start() and stop(). Read a vehicle type from the user ("Car" or "Bike"), create the matching object using a Vehicle reference, and call start() followed by stop() to demonstrate interface implementation. If the input doesn't match either type, print "Invalid vehicle type".

Input:

A single string, the vehicle type: "Car", "Bike", or any other value.

Output:

Two lines showing the start and stop messages of the matching vehicle, or "Invalid vehicle type" if the input doesn't match.

Example:

Input:

Car

Output

Car started

Car stopped

Q14. Reverse a String Using StringBuilder

You are adding a text utility feature to a note-taking app. One of the requested features lets a user type any word or sentence and instantly see it reversed, which the app implements efficiently using Java's built-in StringBuilder class instead of manual character swapping.

Problem Statement:

Write a Java program that reads a string and reverses it using the reverse() method of the StringBuilder class. Print the reversed string.

Input:

A single string s (may contain letters, digits, or spaces).

Output:

The reversed string.

Example:

Input:

hello

Output

olleh

Q15. Row-wise Sum in a 2D Array

You are building a simple gradebook utility for a teacher. Each row of a 2D array holds a student's marks for all subjects, and the teacher wants to quickly see each student's total marks without adding them up by hand.

Problem Statement:

Write a Java program that reads a 2D array of integers with a given number of rows and columns, and prints the sum of each row on its own line.

Input:

Line 1: two integers, rows and cols. Next "rows" lines: "cols" integers each, the elements of that row.

Output:

"rows" lines, each containing the sum of the corresponding row.

Example:

Input:

2 3

1 2 3

4 5 6

Output

6

15